

PROGRAMAÇÃO ORIENTADA A OBJETOS

KiOferta

Encapsulamento e Herança

Atividade da Aula 5 · Parte B · Análise e Desenvolvimento de Sistemas

PARTE 01

O projeto e a arquitetura

O que é o KiOferta

Backend em **FastAPI** que usamos como estudo de caso na disciplina. O módulo de **produtos** (GET /api/produtos) já estava pronto.

- **Parte A:** criar o login (POST /api/login) no mesmo padrão de produtos.
- **Parte B:** mostrar onde usamos encapsulamento e herança no projeto.

Como funciona o login

O app envia **nome** e **senha**.
A API devolve o perfil e as **permissões** do usuário.

As permissões vêm da classe de cada usuário (herança, aula 5).

Arquitetura em camadas



ID	NOME	SENHA	PERFIL
1	bia	bia123	visitante
2	ana	ana123	contribuidor
3	caio	caio123	moderador

app/data/usuarios_mock.py

São 3 usuários, cada um com um perfil. Cada perfil vira uma **classe**.

Esses dados são fixos e não podiam ser alterados.

PARTE 02

Encapsulamento

Atributos protegidos

Os atributos começam com `_` e são lidos ou alterados só pelos métodos da própria classe.

CLASSE	ATRIBUTOS PROTEGIDOS	MÉTODOS DE ACESSO
Produto	<code>_id</code> <code>_nome</code> <code>_categoria</code> <code>_preco</code>	<code>mostrar_*</code> · <code>alterar_nome</code> · <code>alterar_preco</code>
Mercado	<code>_id</code> <code>_nome</code> <code>_lat</code> <code>_lng</code>	<code>mostrar_*</code> · <code>alterar_nome</code> · <code>alterar_localizacao</code>
Oferta	<code>_id</code> <code>_produto</code> <code>_mercado</code> <code>_novo_preco</code>	<code>mostrar_*</code> · <code>alterar_preco</code>
Usuario	<code>_id</code> <code>_nome</code> <code>_senha</code>	<code>mostrar_id</code> · <code>mostrar_nome</code> · <code>alterar_*</code>

Validação nos métodos

app/models/produto.py

```
def __init__(self, id, nome, categoria, preco):
    self._id = id
    self._categoria = categoria
    self.alterar_nome(nome)
    self.alterar_preco(preco)

def alterar_preco(self, novo_preco):
    if novo_preco < 0:
        raise ValueError('preço não pode ser negativo')
    self._preco = novo_preco
```

- **Produto:** o `__init__` chama `alterar_nome` e `alterar_preco`, então a validação vale também na criação.
- **Mercado:** latitude entre -90 e 90, longitude entre -180 e 180.
- **Oferta:** preço maior que zero e menor que o preço normal do produto.

A Oferta pega o preço com `produto.mostrar_preco()`, sem acessar `produto._preco` direto.

PARTE 03

A senha e os controllers

A senha

app/models/usuario.py

```
def alterar_senha(self, nova_senha):  
    if len(nova_senha) < 4:  
        raise ValueError('senha precisa ter  
                           pelo menos 4 caracteres')  
    self._senha = nova_senha  
  
def conferir_senha(self, senha):  
    return self._senha == senha  
  
# não existe mostrar_senha()
```

Não existe método que retorne a senha. De fora só dá para conferir se ela está certa, com `conferir_senha()`.

O `LoginController` valida o login sem `acessar_senha`.

Obs.: no mock a senha está em texto puro. Se um dia usarmos hash, só o `conferir_senha()` precisa mudar.

Encapsulamento no controller

app/controllers/login_controller.py

```
class LoginController:
    def __init__(self):
        self._usuarios = carregar_usuarios()

    def autenticar(self, nome, senha):
        for usuario in self._usuarios:
            if usuario.chama_se(nome) and \
               usuario.conferir_senha(senha):
                return self._para_dicionario(usuario)
        return None
```

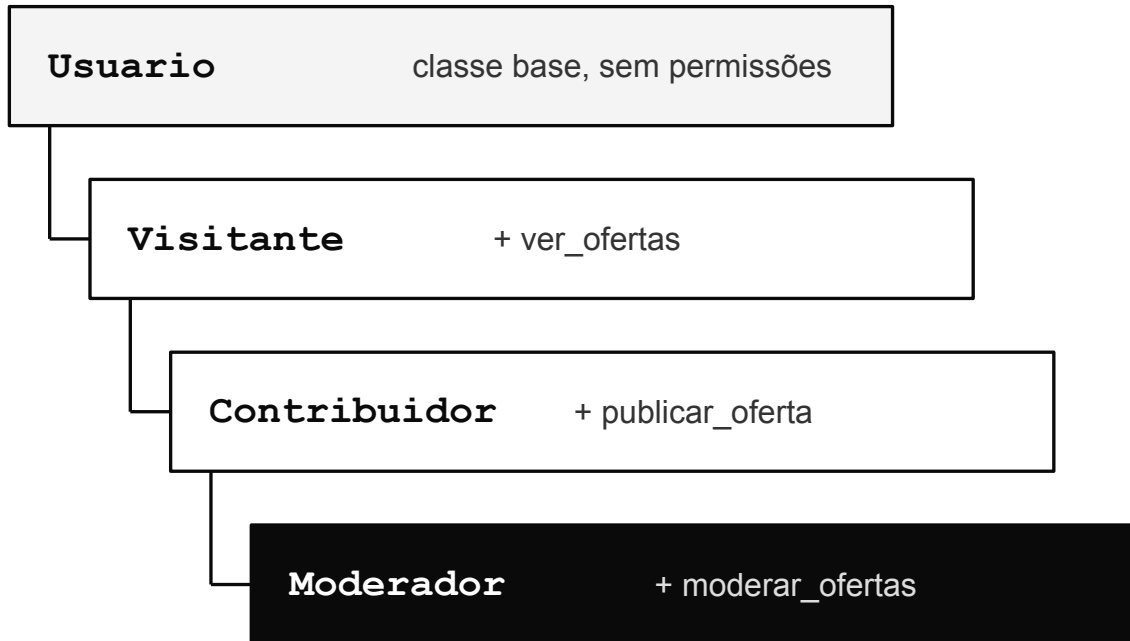
- `self._usuarios` e `self._produtos` são protegidos.
- `_para_dicionario()` é um método interno.
- Quem usa o controller só chama `autenticar()` ou `listar()`.

Rotas e controllers usam só os métodos públicos dos objetos.

PARTE 04

Herança no módulo de login

Hierarquia de usuários



Cada classe herda da anterior e **sobrescreve só o método da permissão nova**.

O **Moderador** é um **Contribuidor**, que é um **Visitante**. Por isso ele tem as três permissões.

Métodos sobrescritos

```
class Usuario:
    def pode_ver_ofertas(self):
        return False
    def pode_publicar_oferta(self):
        return False
    def pode_moderar_ofertas(self):
        return False
```

```
class Visitante(Usuario):
    def pode_ver_ofertas(self):
        return True

class Contribuidor(Visitante):
    def pode_publicar_oferta(self):
        return True

class Moderador(Contribuidor):
    def pode_moderar_ofertas(self):
        return True
```

Na classe base tudo retorna `False`. Se alguém criar um perfil novo e não configurar, ele começa sem nenhuma permissão.

Permissões e criação dos objetos

Usuario.listar_permissoes()

```
def listar_permissoes(self):
    permissoes = []
    if self.pode_ver_ofertas():
        permissoes.append('ver_ofertas')
    if self.pode_publicar_oferta():
        permissoes.append('publicar_oferta')
    if self.pode_moderar_ofertas():
        permissoes.append('moderar_ofertas')
    return permissoes
```

app/models/usuario.py

```
PERFIS = {
    'visitante': Visitante,
    'contribuidor': Contribuidor,
    'moderador': Moderador,
}

def carregar_usuarios():
    return [PERFIS[u['perfil']](
        u['id'], u['nome'], u['senha'])
        for u in USUARIOS]
```

`listar_permissoes()` fica só em `Usuario` e serve para as três subclasses. O perfil em texto só é usado no `PERFIS`.

PARTE 05

Polimorfismo e demonstração

Polimorfismo

```
def _para_dicionario(self, usuario):  
    return {  
        'id': usuario.mostrar_id(),  
        'nome': usuario.mostrar_nome(),  
        'perfil': usuario.mostrar_perfil(),  
        'permissoes':  
            usuario.listar_permissoes(),  
    }
```

LOGIN	CLASSE	PERMISSOES
bia	Visitante	ver_ofertas
ana	Contribuidor	ver_ofertas publicar_oferta
caio	Moderador	ver_ofertas publicar_oferta moderar_ofertas

O controller chama os mesmos métodos para qualquer usuário. A resposta muda de acordo com a classe.

Demonstração: POST /api/login

terminal

```
uvicorn main:app --reload  
# abrir http://127.0.0.1:8000/docs
```

requisição

```
{"nome": "caio", "senha": "caio123"}
```

senha errada

```
{"nome": "caio", "senha": "errada"}
```

200 OK

```
{  
  "id": 3,  
  "nome": "caio",  
  "perfil": "moderador",  
  "permissoes": [  
    "ver_ofertas",  
    "publicar_oferta",  
    "moderar_ofertas"  
  ]  
}
```

401 Unauthorized

```
{"detail": "nome ou senha inválidos"}
```

Resumo

Encapsulamento

Atributos com `_`, validação nos métodos e senha sem getter.

Herança

Usuario, Visitante, Contribuidor e Moderador, cada um somando uma permissão.

Polimorfismo

O controller trata todos os usuários do mesmo jeito.

Login e produtos seguem a mesma estrutura: **mock, model, controller e rota**.

PROGRAMAÇÃO ORIENTADA A OBJETOS

Obrigado! Perguntas?